

Documento Técnico

Definición Dirigida por Sintaxis para un lenguaje tipo C

Integrantes:

1. Mendez Camacho Monserrat - 424490034
2. Rodríguez Nevarez Edwin - 312227441
3. Carrillo Gonzalez Luis Israel - 321050324
4. Reyna Méndez Cristian Ignacio - 320140579

Grupo: 2026-2

Junio 2026

Índice

1. Descripción del lenguaje	3
1.1. Tokens	3
1.2. Sintaxis	3
1.3. Características soportadas	3
2. Gramática	4
2.1. Gramática utilizada	4
2.2. Explicaión de conflictos si existen	5
2.3. Precedencia y Asociatividades	6
3. Análisis Semántico	6
3.1. Atributos	6
3.2. Reglas Semánticas	6
3.3. Manejo de Ámbitos	7
3.4. Manejo de Tipos	7
3.5. Verificación Semántica	7
4. Tablas	7
4.1. Tabla de Símbolos (SymTab)	7
4.2. Tabla de Tipos (TypeTab)	8
4.3. Temporales y Etiquetas (CodeGen)	8
5. Casos de Prueba	9

1. Descripción del lenguaje

1.1. Tokens

Para empezar, usamos una herramienta llamada Flex que se encarga de leer el código fuente (texto puro) y agrupar los caracteres en “pedacitos” que tengan sentido para nosotros. A estos pedazos los llamamos **tokens**. Los tokens que nuestro lenguaje sabe reconocer son:

- **Palabras reservadas:** Son las palabras clave que el lenguaje ya tiene apartadas para que funcionen como comandos especiales. En nuestro proyecto usamos: `def` (para definir funciones), los tipos de datos como `int`, `float`, `bool`, `char`, además de `struct` para agrupar variables. También tenemos palabras para controlar cómo avanza el programa, como `if`, `else`, `while`, `for`, `break`, `return`, y los valores lógicos `true` y `false`.
- **Identificadores (ID):** Son los nombres que el usuario inventa para llamar a sus variables, funciones o estructuras. La única regla es que siempre deben empezar con una letra o un guion bajo, y después de eso ya pueden llevar más letras o números.
- **Literales (Los valores directos):** Son los datos escritos directamente en el código. Reconocemos números enteros sin decimales (ej. 15), números con decimales (ej. 3.14), y caracteres individuales que van entre comillas simples (ej. 'a').
- **Operadores:** Las matemáticas (+, -, *, /, %), las comparaciones para saber si algo es mayor o igual (<, >, <=, >=, ==, !=), las compuertas lógicas (&&, ||) y el signo de igual para asignar valores (=).
- **Símbolos de puntuación:** Usamos el punto y coma ; para decir que una instrucción terminó, comas , para separar listas de cosas, paréntesis () , llaves {} para meter código dentro de bloques, corchetes [] para los arreglos, y el punto . para entrar a ver qué hay dentro de un struct.

1.2. Sintaxis

Una vez que Flex nos separó el texto en palabras con sentido (tokens), usamos otra herramienta llamada Bison. Bison se encarga de revisar que esas palabras estén ordenadas de una forma que tenga lógica.

Nuestro lenguaje funciona de forma muy parecida a C. Un programa válido en nuestro lenguaje es básicamente una lista de cosas globales. En la parte de afuera del programa podemos declarar variables, armar **structs** para agrupar datos, o crear funciones.

Dentro de esas funciones es donde escribimos el comportamiento real. Ahí podemos usar ciclos como los `while` o `for` para repetir cosas, y condicionales como el `if` para que el programa tome decisiones.

1.3. Características soportadas

Hicimos que el compilador abarcara bastantes cosas interesantes. Las principales capacidades de nuestro lenguaje son:

- Guardar información en variables de tipos clásicos: números enteros (`int`), números con decimales (`float`), letras sueltas (`char`) o valores de verdad (`bool`).
- Crear arreglos, incluso de varias dimensiones (matrices), y armar tus propios tipos de datos complejos metiendo varias variables distintas adentro de un `struct`.
- Declarar tus propias funciones, mandarles datos como parámetros y hacer que te regresen un resultado útil.
- Crear variables locales o variables globales.
- Manejar el flujo de lo que hace el programa usando bucles y condicionales.
- **El objetivo principal:** Nuestro compilador lee todo este código y, si está bien escrito, genera Código Intermedio de Tres Direcciones (TAC). Es un código especial, más simple, que es el paso justo anterior a convertir todo código de máquina o ensamblador.

2. Gramática

2.1. Gramática utilizada

Para que el compilador sepa si el código que escribimos está bien estructurado, utiliza las reglas de producción:

- **El Programa Completo (P y H):**

- $P \rightarrow H$
- $H \rightarrow D H \mid \epsilon$

Todo programa empieza en la regla **P**. Básicamente decimos que un programa es una lista de declaraciones globales (**H**), o bien, que puede estar vacío (ϵ).

- **Declaraciones Globales (D):**

- $D \rightarrow T L ; \mid \text{struct } ID \{ H \} ; \mid \text{def } T ID (F) \{ H R \}$

Con estas reglas podemos declarar variables (**T L ;**), armar un `struct` para agrupar variables, o definir funciones completas usando la palabra `def`.

- **Tipos de Datos (T, B y A):**

- $T \rightarrow B A$
- $B \rightarrow \text{int} \mid \text{float} \mid \text{bool} \mid \text{char} \mid \text{struct } ID$
- $A \rightarrow [\text{INTLIT}] A \mid \epsilon$

Los tipos base (**B**) soportan variables simples (`int`, `float`, etc.) y estructuradas. A través de la regla **A**, a cualquiera de estos tipos les podemos agregar corchetes `[]` sucesivos para convertirlos en arreglos de una o múltiples dimensiones.

- **Listas de variables o parámetros (L, F y G):**

- $L \rightarrow L , ID \mid ID$
- $F \rightarrow G \mid \epsilon$
- $G \rightarrow G , T ID \mid T ID$

Estas reglas permiten separar por comas las variables que declaramos en una misma línea (**L**), o la lista de parámetros que le enviamos a una función (**F** y **G**).

■ **El interior de las funciones - Sentencias (R y S):**

- $R \rightarrow S R \mid \epsilon$
- $S_ASIG \rightarrow ID = E \mid C = E \mid Z = E$
- $M_IF \rightarrow \text{if} (E)$
- $S \rightarrow \text{break} ; \mid \text{return } E ; \mid S_ASIG ; \mid M_IF S \mid M_IF S \text{ else } S \mid \text{while} (E) S \mid \text{for} (S_ASIG ; E ; S_ASIG) S \mid \{ R \}$

Aquí se establece qué es una instrucción válida (**S**). Puede ser una asignación de datos (**S_ASIG**), estructuras condicionales (**if**, **else**), ciclos de control (**while**, **for**), comandos de escape (**break**, **return**) o bloques de código rodeados por llaves.

■ **Expresiones Matemáticas y Lógicas (E):**

- $E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid E \% E \mid E \mid \mid E \mid E \&\& E \mid E > E \mid E < E \mid E >= E \mid E <= E \mid E == E \mid E != E \mid (E) \mid C \mid Z \mid ID \mid ID (N) \mid INTLIT \mid FLOATLIT \mid TRUE \mid FALSE \mid CHARLIT$

Todo tipo de cálculos: sumas, restas, divisiones, evaluación booleana, y valores como números o literales.

■ **Accesos a Arreglos, Structs y Argumentos (N, M, C y Z):**

- $N \rightarrow M \mid \epsilon$
- $M \rightarrow M , E \mid E$
- $C \rightarrow ID [E] \mid C [E]$
- $Z \rightarrow ID . ID \mid Z . ID$

Las reglas **N** y **M** procesan los argumentos cuando queremos llamar una función. La regla **C** es para indexar casillas de un arreglo (**arreglo[2]**), y la regla **Z** le permite usar la notación de punto para acceder a los campos internos de un struct (**persona.edad**).

2.2. Explicaión de conflictos si existen

Tuvimos el problema del *else*. ¿Qué pasa si el usuario pone un **if** adentro de otro **if**, y luego pone un solo **else**? El analizador entra en conflicto porque no sabe a cuál de los dos **if** le pertenece ese **else**.

Para solucionarlo, en nuestro archivo de Bison incluimos la instrucción `%expect 1`. Esto le indica a Bison que ignore esa advertencia y la resuelva con desplazamiento, lo que significa que el **else** se emparejará siempre con el **if** más cercano, evitando problemas de ambigüedad.

2.3. Precedencia y Asociatividades

Para evitar que una ecuación como $2 + 3 * 4$ produzca resultados erróneos (sumar antes de multiplicar), le dimos un orden de jerarquía estricto a los operadores. El analizador los agrupa siempre resolviendo primero los de mayor jerarquía. Todos tienen asociatividad por la izquierda.

La lista desde el de menor importancia al de mayor importancia es la siguiente:

1. **Operadores Lógicos:** El O lógico ($||$) seguido por el Y ($\&\&$).
2. **Igualdades:** Verificación de exactitud ($==$) o diferencia ($!=$).
3. **Relacionales:** Verificación de magnitudes ($>$, $<$, $>=$, $<=$).
4. **Sumas y Restas:** ($+$, $-$).
5. **Multiplicación, división y módulo:** ($*$, $/$, $\%$). Tienen la mayor prioridad.

3. Análisis Semántico

3.1. Atributos

Para que nuestro analizador pueda revisar la lógica, necesita recordar datos de cada variable o expresión mientras recorre el árbol sintáctico. Para esto usamos “atributos” que viajan a través de los nodos. Los principales son:

- **tipo:** Es un número (ID) que indica qué tipo de dato es la expresión (p.ej. si es un `int` o un `float`).
- **base:** Guarda de qué está compuesto un arreglo o struct. Por ejemplo, para un arreglo de enteros, su tipo base será `int`.
- **dir:** Guarda la dirección temporal o el nombre de la variable (ej. `t1`, `t2`) donde el compilador asignó el resultado de una operación.
- **tam:** Indica el tamaño de la variable (en bytes), para calcular los desplazamientos o *offsets* en arreglos y structs.
- **baseStr:** Es el nombre original de una estructura o arreglo, usado para concatenar y armar el código intermedio (TAC) final.

3.2. Reglas Semánticas

Su trabajo es vigilar que todo lo que programó el usuario sea legal. Con un enfoque de **Definición Dirigida por Sintaxis**, las reglas se encargan de:

1. **Propagar la información:** Usamos variables auxiliares globales (como `currentType`) para recordar el tipo de dato que se está leyendo en las declaraciones.
2. **Casteo automático:** Si una operación combina un `int` con un `float`, nuestras reglas ponen instrucciones extra para ampliar (`inttofloat`) o reducir (`floattoint`) los valores y mantener la integridad.

3. **Generar saltos:** Para que las estructuras condicionales (`if`, `while`) funcionen a bajo nivel, usamos funciones como `CodeGen::newLabel()` que crean etiquetas (`L0`, `L1`) y con instrucciones `goto` se brinca si las condiciones booleanas no se cumplen.

3.3. Manejo de Ámbitos

Para resolver la ambigüedad de las variables locales y globales usamos una estructura llamada `PilaTs` (Pila de Tablas de Símbolos).

- El fondo de la pila (`bottom()`) almacena las variables y funciones globales, visibles en todo el programa.
- La punta de la pila (`top()`) resguarda las variables locales que solo existen dentro del bloque actual.
- Cada vez que el analizador entra al cuerpo de una función nueva o de un `struct`, apila un entorno nuevo. Al salir, lo desapila y destruye esas variables locales.

3.4. Manejo de Tipos

Para registrar los tipos usamos `TypeTab`. Ésta ya nace con los tipos primitivos precargados (`int`, `float`, `bool`, `char`).

Si el código declara un nuevo tipo compuesto (arreglos o `structs`), `TypeTab` lo registra en su catálogo asignándole un ID, anotando su tamaño total, con su tipo base y, en el caso de los `structs`, dándole una tabla de símbolos nueva para sus variables internas.

3.5. Verificación Semántica

Implementamos validaciones lógicas:

- **Unicidad:** Se lee la tabla local para ver que no se declaren dos variables, `structs` o funciones con el mismo identificador en el mismo ámbito.
- **Existencia:** Cuando se usa una variable, se revisa primero el `top()` y si no está, el `bottom()`, para confirmar que ya haya sido declarada.
- **Compatibilidad estricta:** Con la función `compatibles(t1, t2)`, el sistema no deja operaciones raras (como sumar un `struct` con un `int`).
- **Uso de arreglos:** Cualquier indexación de un arreglo (`a[x]`) se haga usando un número entero (`int`) y que este sea mayor estricto a 0.
- **Integridad de retorno:** Confirma que la variable retornada por el `return` sea exactamente el tipo de dato que la función prometió devolver (`tipoReturnFunc`).

4. Tablas

4.1. Tabla de Símbolos (`SymTab`)

Esta tabla funciona como el registro del programa. Por cada variable, arreglo, `struct` o función que el usuario declara, la `SymTab` anota:

- **Identificador:** El nombre que le asignó el programador.
- **Dirección de memoria:** Su dirección relativa o *offset* (`dir`) para saber en qué byte exacto va a vivir esa información.
- **Tipo:** Un número ID que nos sirve para consultar qué tipo de dato es en la tabla de tipos.
- **Categoría:** si es una variable local `var`, un argumento `arg`, una función `func`, o un `struct`.
- **Lista de argumentos:** Si se trata de una función, guarda el orden de los tipos de parámetros que pide, para evitar que le envíen datos que no son.

4.2. Tabla de Tipos (TypeTab)

Mientras que la tabla de símbolos guarda los nombres de las variables, la tabla de tipos es dice las propiedades físicas de los datos:

- Un identificador único.
- Su nombre textual (ej. `array`, `float`).
- Su peso o tamaño exacto en bytes (`tam`).
- Si es un arreglo, anota el tipo base.
- Si es un `struct`, guarda un apuntador hacia una Tabla de Símbolos local, ya que un `struct` es un micromundo con sus propias variables.

4.3. Temporales y Etiquetas (CodeGen)

A la hora de hacer el Código de Tres Direcciones (TAC) final, nuestro generador de código (CodeGen) debe partir las operaciones grandes. Para eso, inventa e incrementa de manera secuencial:

- **Temporales:** Variables desechables donde se almacenan cálculos intermedios. Las nombra automáticamente como `t0`, `t1`, `t2`...
- **Etiquetas:** Puertas de destino para los saltos condicionales (`goto`). Las nombra como `L0`, `L1`, `L2`...

Para coordinar a qué etiqueta exacta le corresponde saltar a un `break` cuando anidamos muchos ciclos (como un `for` dentro de un `while` dentro de un `if`), utilizamos cinco pilas auxiliares (`pilaBreak`, `pilaReturn`, `pilaTrue`, `pilaFalse` y `pilaDir`). Éstas hacen que el código viaje al nivel correcto sin colapsar.

5. Casos de Prueba

Se hicieron pruebas en la carpeta `pruebas/` del repositorio. Estos nos muestran todos los posibles casos:

- **Programas Válidos** (`programas_validos/`): Se probaron 5 archivos que tienen desde cálculos simples hasta el uso de matrices y structs. Todos generan Código Intermedio (TAC) de manera correcta.
- **Errores Léxicos** (`prog_errores_lexicos/`): 3 pruebas con caracteres que no se aceptan (como \$ o #). El escáner (Flex) los detecta y sale de inmediato.
- **Errores Sintácticos** (`prog_errores_sintacticos/`): 3 pruebas en las que se abrieron llaves sin cerrarlas, se omitieron puntos y comas, o se hicieron expresiones sin sentido. Bison atrapa los conflictos de estructura.
- **Errores Semánticos** (`prog_errores_semanticos/`): 3 pruebas en las que se intentó indexar un arreglo usando números decimales, sumar variables no declaradas o de tipos incompatibles. Las reglas semánticas detienen la compilación.